

Transfer Reinforcement Learning aided Distributed Network Slicing Optimization in Industrial IoT

Tianle Mai, *Student Member, IEEE*, Haipeng Yao, *Senior Member, IEEE*, Ni Zhang, Wenji He, *Student Member, IEEE*, Dong Guo, and Mohsen Guizani, *Fellow, IEEE*

Abstract—With the growth of the number of IoT devices and the emergence of new applications, satisfying distinct QoS in the same physical network becomes more challenging. Recently, with the advance of network functions virtualization (NFV) and software-defined networking (SDN) technologies, the network slicing technique has emerged as a promising solution. It can divide a physical network into multiple virtual networks, therefore providing different network services. In this paper, to meet distinct QoS in industrial IoT, we design a network slicing architecture over the SDN-based Long-Range Wide Area Network (LoRaWAN). The SDN controller can dynamically split the network into multiple virtual networks according to different business requirements. On this basis, we proposed a Deep Deterministic Policy Gradient (DDPG) based slice optimization algorithm. It enables LoRa gateways to intelligently configure slice parameters (e.g., Transmission Power, Spreading Factor) to improve the slice performance in terms of QoS, energy efficiency, and reliability. In addition, to accelerate the training process across multiple LoRa gateways, we leverage the transfer learning framework and design a transfer learning-based multi-agent DDPG (TMDDPG) algorithm.

Index Terms—Network Slicing, Transfer Learning, Industrial IoT, Multi-agent Reinforcement Learning.

I. INTRODUCTION

INDUSTRIAL IoT as the most powerful and exciting technology, has quickly become a disruptive force reshaping how we live and work [1]. Massive industrial devices are now connected to the network, allowing for data collecting, exchanging, and analyzing. According to the Juniper's report, there will be 37 billion industrial IoT devices by 2025. Faced with such massive devices, IoT network technology is now confronted with unprecedented challenges.

Currently, the industrial IoT network market is dominated by the Long-Range Wide Area Network (LoRaWAN) technology [2]. It is a non-cellular wireless wide area network technology, which uses the LoRa radio modulation technology

to provide low cost, low power, and long-range communication. Such appealing characteristics enable LoRaWAN suited for a wide range of industrial applications, particularly in distant areas where businesses like petroleum drilling, mining, and construction operate. However, with the emergence of new applications, providing the QoS to different IoT devices becomes a critical challenge in LoRaWAN [3].

Recently, network slicing technology has emerged as a viable solution to address this challenge. Network slicing is the process of dividing a single physical network into many logical (virtual) networks using network virtualization (i.e., NFV and SDN) [4]. Different slices have different logical topology, security rules, and performance characteristics for the sake of fulfilling different business purposes. In this paper, we design a network slicing architecture over SDN based LoRaWAN, where the SDN controller can dynamically partition LoRa gateways' resources (e.g., physical channel) into several virtual networks on the fly.

Meanwhile, considering the limited resources on each gateway, designing an efficient slice resource optimization scheme is another crucial problem. The gateway should be able to configure slice parameters (e.g., Bandwidth (BW), Spreading Factor (SF), Transmission Power (TP)) to satisfy the distinct QoS [5]. To address this issue, we propose a DDPG based slice resource optimization algorithm [6]. The LoRa gateways using the DDPG are able to improve the performance by exploring the environment and learning directly from their experiences.

While such learning mechanism can converge to the optimal policy, in the end, it has to take a large number of training episodes. Especially in our scenario, each slice agent on each LoRa gateway has to learn from scratch (i.e., randomized policy), thereby resulting in a long learning time to reach the system's optimal performance. To accelerate the training process, we introduce the transfer learning framework [7]. Transfer learning is a machine learning technique where experience gained acquired from one task can be transferred to other related tasks. Therefore, to accelerate the learning process across multiple LoRa gateways, we propose a Transfer learning-based Multi-agent DDPG (TMDDPG) algorithm.

The major contributions of this paper are summarized as follows.

- We design a network slicing architecture over the SDN-based LoRaWAN, where the SDN controller dynamically split physical resources into multiple virtual networks.
- We discuss the network slice optimization problem. A DDPG based slice optimization algorithm is designed for the sake of searching the optimal slice's parameters

T. Mai, H. Yao, and W. He are with State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China. E-mail: yaohaipeng@bupt.edu.cn, macheal-mai@gmail.com, hewenji@bupt.edu.cn.

N. Zhang is with Sixth Research Institute of China Electronic Corporation, Beijing, China. Email: Zhangni@ncse.com.cn.

D. Guo is with School of Information and Electronics, Beijing Institute of Technology, Beijing 100081, China. E-mail: 7520190141@bit.edu.cn.

M. Guizani is with Machine Learning Department, MBZUAI, Abu Dhabi, UAE. E-mail: mguizani@gmail.com.

This research was supported by the funding from the National Key R&D Program of China under grant 2018YFB1800805, Artificial Intelligence and Smart City Joint Laboratory (BUPT-TGSTII) (B2020001), Future Intelligent Networking and Intelligent Transportation Joint Laboratory (BUPTCTIC) (B2019007), Intelligent Network Joint Laboratory (BUPTIN) (B2021006), and the BUPT Excellent Ph.D Student Foundation under Grant CX2020108.

configuration.

- To accelerate the learning process across multiple LoRa gateways, we proposed a transfer learning-based multi-agent DDPG (TMDDPG) algorithm.

The rest of this paper is organized as follows. In Section II, we first review the related work. In Section III, we design a network slicing architecture over SDN based LoRaWAN network and present the system model. We design the TMDDPG based slices optimization algorithm in Section IV. In Section V, we present the simulation results. Finally, Section VI discusses the conclusion.

II. RELATED WORK

Recently, a considerable amount of literature has been published on network slicing in LoRa-based networks. In [8], Afolabi *et al.* presented a comprehensive survey on network slicing, and discussed the research challenges of these technologies. In the following, we will briefly discuss related works from the perspective of the network slicing and the resource allocation.

A. Network Slicing

Much of the current literature on LoRa-based networks pays particular attention to network slicing optimization schemes. During the design of the algorithm, it involves three problems, including 1) slicing admission and definition problem; 2) Inter-slicing resources reservation; 3) Intra-slice resources optimization [9]. In [10], Dawaliby *et al.* formulated the slicing admission and definition as a one-to-many matching game, the inter-slicing problem as a Bankruptcy resource reservation game, and the intra slice resource allocation as a multi-objective optimization problem. In [8], the author uses the maximum likelihood estimation (MLE) algorithm to dynamically reserve LoRa gateways' bandwidth resource for each slice (i.e., inter-slicing resources reservation). In [11], the author proposed a GMM and TOPSIS algorithm to optimize intra-slice resources (i.e., SF and TP configuration). The experiment result shows that the TOPSIS can effectively enhance slice utility in terms of throughput, reliability, and delay. In [10], Messaoud *et al.* proposed a deep federated Q-Learning (DFQL) based dynamic slices approach to maximize self-QoS requirements. In this paper, considering that LoRaWAN is mainly used in fixed IoT devices, we assume the previous two problems can be well solved by a general solution (e.g., convex optimization method). Therefore, we only focus on the intra-slice resources allocation problem.

B. Resource Allocation

Recently, a large and growing body of literature has investigated the resource allocation problem in LoRaWAN. Researchers always employ the Adaptive Data Rate (ADR) scheme to minimize energy consumption and maximize throughput by adjusting the SF and TP configuration. In [12], Kufakunesu *et al.* present a comprehensive survey on ADR Optimization in LoRaWAN. In [13], Ilahi *et al.* proposed a LoRaDRL algorithm. It leverages the deep reinforcement learning algorithm to dynamically optimize parameters configuration.

In [14], Park *et al.* used the DRL algorithm to address the resources allocation problem in LoRaWAN. The experiment results demonstrate that the DRL-based scheme is 15% better than the current ADR algorithm in terms of throughput and energy efficiency. In [15], Reynders *et al.* proposed a two-step lightweight SF-TP scheduling scheme. The simulation results demonstrate that the proposed scheme can effectively improve the reliability and scalability of LoRaWAN.

III. SYSTEM MODEL

In this section, we design a SDN-based network slicing architecture in LoRaWAN. Then, we present the system model and problem formulation of slice optimization.

A. Network Slicing Architecture

A typical LoRaWAN architecture consists of end devices, LoRa gateways, network servers, and application servers. The gateways are responsible for forwarding messages from end devices to network servers. The network servers are responsible for network management functions, including Over-The-Air-Activation, message routing, acknowledgment of messages, and adaptive data rate control. The application servers are used to process application-specific data messages received from end devices.

In this paper, we leverage SDN and NFV technologies to enhance the flexibility and programmability of the LoRaWAN. As shown in Fig. 1, we present a network slicing architecture over SDN-based LoRaWAN. The architecture runs on open-spec commodity compute and networking hardware, and connects with the LoRa gateways. All the network functions are built on micro-services, hosted into containers, and orchestrated with Kubernetes. In our architecture, it contains three layers, named Application Plane, Control Plane, and Infrastructure Plane.

Application Plane: In this plane, the applications can issue their QoS requirements and desired network behavior to the control plane via northbound application program interfaces. Meanwhile, it can leverage network information (e.g., network topology, network state) for its internal decision-making purposes. According to different QoS, the applications can be classified into three types, including best effort (BE), reliability aware (RA), and urgency and reliability aware (UR). The URA application requires the highest priority. Examples include emergency alerting and robot arm control. The RA applications require lower priority (e.g., security systems), while the BE require the lowest priority (e.g., smart metering applications) [16].

Control Plane: The control plane is composed of multiple SDN controllers. The controller can translate the application requirements to the infrastructure plane and provide global network abstractions of the LoRaWAN to the application plane [17]. According to the complete knowledge of network state and applications QoS, the controller can split physical network resources (e.g., channels on LoRa gateways) for different slices, and assign IoT devices to the corresponding slice.

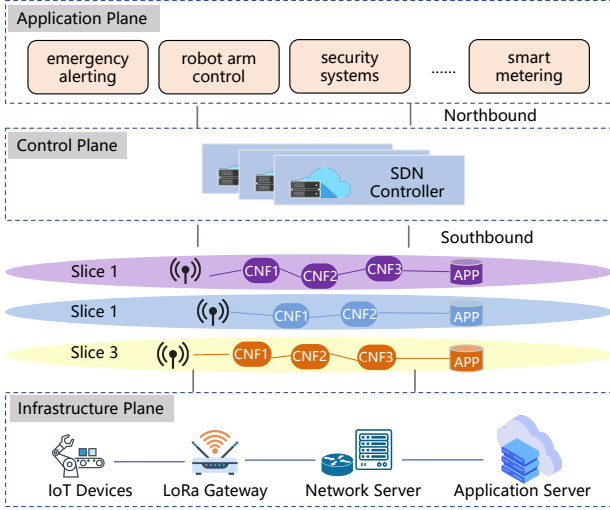


Fig. 1. Network Slicing Architecture over SDN-based LoRaWAN

Infrastructure Plane: The infrastructure plane consists of end devices, LoRa gateways, network servers, and applications servers. The LoRa gateways are connected in a star of stars network topology. And the network servers, and applications servers are hosted into containers. Each gateway will reserve channels resources for each slice according to the configuration issued by the SDN controller. And the container-based network server will be dynamically orchestrated according to controller commands [18].

B. System Model

Consider a network consisting of a set of LoRa gateways $\mathcal{G} = \{1, \dots, G\}$, and a set of industrial IoT devices $\mathcal{D} = \{1, \dots, D\}$. We assume that each device will be assigned to a specified network slice $j \in \mathcal{J} = \{1, \dots, J\}$ of the closest gateway according to its QoS. Each gateway will reserve its channel resource $\mathcal{C} = \{1, \dots, C\}$ for each slice, where $c_{j,g}$ denote the channel associated to the slice j on the gateway g . Each channel has its corresponding bandwidth $b \in \mathcal{B} = \{1, \dots, B\}$. To evaluate the slice performance, we design three evaluation metrics, named Throughput&Delay Metric, Energy Efficiency Metric, and Reliability Metric. We will detail the design in the following.

1) Throughput&Delay Metric: The first evaluation indicator is throughput and delay. In LoRaWAN, each device adopts a specific spreading factor (varying between 7 and 12) for message transmission, which means each symbol will be encoded into 2^{SF} signals (chips). Thus, the LoRa modulation data rate can be formulated as:

$$r_d = SF \cdot \frac{b_{j,g}}{2^{SF}} \cdot CR \quad \text{bits/s}, \forall d \in \mathcal{D}_{j,g}, \quad (1)$$

where r_d denotes the LoRa modulation bit rate of device d , CR denotes the code rate, and $b_{j,g}$ denotes the bandwidth assigned for slice j on the gateway g . Then, the transmission delay can be expressed as:

$$t_d = \frac{L}{r_d} \quad \text{seconds}, \forall d \in \mathcal{D}_{j,g}, \quad (2)$$

where L is the packet length. Based on this, we define a QoS metric $u_{QoS}^{\mathcal{D}_{j,g}}$ to measure the delay and throughput of slice j , which can be formulated as:

$$u_{QoS}^{\mathcal{D}_{j,g}} = \sum_{d \in \mathcal{D}_{j,g}} (\bar{r}_d + (1 - \bar{t}_d)), \quad (3)$$

where $\bar{r}_d$ and $\bar{t}_d$ indicate the normalized value of throughput and transmission delay respectively.

2) Energy Efficiency Metric: Another evaluation metric of slice performance is energy efficiency. The energy consumption consists of two states, including active mode consumption $P_{j,g}^{ac}$ and sleep mode $P_{j,g}^{sleep}$. Hence, the energy consumption of slice j during a slicing interval time T can be described as:

$$P_{j,g}^{tt} = P_{j,g}^{ac} T_{active} + P_{j,g}^{sleep} T_{sleep}, \quad (4)$$

where T_{active} is related to the r_d and $P_{j,g}^{ac}$ is related to the transmission power. According to pervious works [19], we formulate the unit power consumption as:

$$u_{EE}^{\mathcal{D}_{j,g}} = \sum_{d \in \mathcal{D}_{j,g}} (1 - \bar{P}_{j,g}^{tt}), \quad (5)$$

where $\bar{P}_{j,g}^{tt}$ is the normalized energy consumption.

3) Reliability Metric: The third evaluation indicator is reliability. Configuring low SF and TP values may cause packet loss due to sensitivity, inter-SF, and intra-SF interference. To evaluate the reliability of transmission, we design the packet success rate metric $u_{REL}^{\mathcal{D}_{j,g}}$, which can be formulated as:

$$u_{REL}^{\mathcal{D}_{j,g}} = \sum_{d \in \mathcal{D}_{j,g}} PSR_{d,j}, \quad (6)$$

$$\text{with } PSR_{d,j} = PR_{d,j}^{tra} + PR_{d,j}^{tre} + PR_{d,j}^{sen},$$

where $PSR_{d,j}$, $PR_{d,j}^{tra}$, $PR_{d,j}^{tre}$ and $PR_{d,j}^{sen}$ are binary variable. The $PR_{d,j}^{tra}$ indicates the packets lost caused by collisions that occur between two end devices configured with the same SF. According to the random access formula, the $PR_{d,j}^{tra}$ can be described as:

$$PR_{d,j}^{tra} = 1 - e^{-2G_{SF}}, \quad (7)$$

where G_{SF} is the number of packets generated when one packet is transmitted.

The $PR_{d,j}^{tre}$ indicates the packets lost due to the inter SF interference. The devices experience a Signal-to-interference-plus-noise ratio (SINR), which can be described as:

$$SINR_{i,j} = \frac{P_i^{rx}}{\sigma^2 + \sum_{n \in \partial_j} P_{n,j}^{rx}}, \quad (8)$$

where P_i^{ac} is the transmission power with $SF = i$, and σ^2 is the white Gaussian Noise. The below matrix denotes the minimum signal power margin threshold with other SF configuration [10].

$$\begin{matrix} & SF_7 & SF_8 & SF_9 & SF_{10} & SF_{11} & SF_{12} \\ \begin{matrix} SF_7 \\ SF_8 \\ SF_9 \\ SF_{10} \\ SF_{11} \\ SF_{12} \end{matrix} & \begin{pmatrix} -6 & 16 & 18 & 19 & 19 & 20 \\ 24 & -6 & 20 & 22 & 22 & 22 \\ 27 & 27 & -6 & 23 & 25 & 25 \\ 30 & 30 & 30 & -6 & 26 & 28 \\ 33 & 33 & 33 & 33 & -6 & 29 \\ 36 & 36 & 36 & 36 & 36 & -6 \end{pmatrix} \end{matrix}$$

TABLE I
LIST OF MAIN NOTATIONS

Parameter	Definition
SF	Spreading Factor
TP	Transmission Power
G	Number of gateways
J	Number of slices
C	Number of channel
D	Number of industrial IoT devices
r_d	The the LoRa modulation data rate of device d
$b_{j,g}$	The bandwidth assigned for slice j on the gateway g
$\bar{P}_{j,g}^{it}$	The the energy consumption of slice j during a slicing interval time T
$PR_{d,j}^{tra}$	The packets lost caused by intra-SF collisions
$PR_{d,j}^{tre}$	The packets lost caused by intre-SF collisions
$PR_{d,j}^{sen}$	The packets lost when a packet is transmitted to the gateways below sensitivity
$u_{QoS}^{\mathcal{D}_{j,g}}$	The satisfaction rate of slice j in terms of delay and throughput
$u_{EE}^{\mathcal{D}_{j,g}}$	The satisfaction rate of slice j in terms of energy efficiency
$u_{REL}^{\mathcal{D}_{j,g}}$	The satisfaction rate of slice j in terms of reliability

Hence, the $PR_{d,j}^{tre}$ can be formulated as:

$$PR_{d,j}^{tre} = \begin{cases} 0, & \text{if packet survives interference} \\ 1, & \text{Otherwise} \end{cases}$$

In addition, the $PR_{d,j}^{sen}$ indicates the packets lost when a packet is transmitted to the gateways below sensitivity, which can be expressed as:

$$PR_{d,j}^{sen} = \begin{cases} 0, & \text{if packet successfully reaches } j \in \mathcal{J} \\ 1, & \text{Otherwise} \end{cases}$$

C. Problem Formulation

At this stage, we can formulate a multi-objective optimization problem. We search for the optimum SF and TP configuration that can simultaneously enhance the slice's QoS, energy efficiency, and reliability. This optimization problem can be formulated as:

$$\max u^{\mathcal{D}_{j,g}} = u_{QoS}^{\mathcal{D}_{j,g}} + u_{EE}^{\mathcal{D}_{j,g}} + u_{REL}^{\mathcal{D}_{j,g}}, \quad (9)$$

subject to the following constraints:

$$C1: d_{j,g} \cap d_{j',g} = \emptyset, \forall j, j' \in \mathcal{J}, \forall g \in \mathcal{G} \quad (10a)$$

$$C2: d_{j,g} \cap d_{j,g'} = \emptyset, \forall j \in \mathcal{J}, \forall g, g' \in \mathcal{G} \quad (10b)$$

$$C3: 0 \leq P_{j,g}^{ac} \leq P^{max}, \quad (10c)$$

$$C4: \sum_{d \in \mathcal{D}_{j,g}} r_d \leq R_{j,g}^{max}. \quad (10d)$$

The constraint $C1$ ensures that one device can only be allocated to one slice. $C2$ ensures that one device can only be allocated to one gateway. $C3$ ensures that the transmission power is limited to the maximum power. And $C4$ ensures that the total transmission power is limited to the maximum rate [20]. As shown in Table I, we list the notations of this paper.

IV. TRANSFER MULTI-AGENT REINFORCEMENT LEARNING

In this paper, we introduce the DDPG algorithm to search the optimal SF and TP parameters of each slice. Then, to accelerate the learning rate, the transfer learning framework is introduced.

A. Markov Decision Process

The slice optimization problem can be described as an independent Markov Decision Process (MDP) [21]. Formally, an MDP can be formalized as a 4-tuple $\langle S, A, \pi, R \rangle$, where S is the state space, A is the action space, Π is the policy space, and R is the immediate rewards. At each step, the slice agent takes an action $a \in A$ according to current policy $\pi(a|s)$ and its observation $s \in S$. Then, the underlying environment will generate an immediate reward R , and the state s will transit to a new state $s' \in S$. Specifically, in our scenario, we will define the three components of MDP in the following.

1) *State Definition*: In this paper, we define the environment state as the device information and packet information. The device information includes spreading factors, transmission power, bandwidth, and energy consumed (ENY). And the packet information includes signal-to-noise ratio (SNR), received signal strength indicator (RSSI), and the total number of packets during a unit time (Num). These values are encoded in an one-hot format, which can be represented as:

$$S = (\bar{SF}, \bar{TP}, \bar{BW}, \bar{ENY}, \bar{SNR}, \bar{RSSI}, \bar{Num}). \quad (11)$$

All the values are average values over an interval period.

2) *Action Definition*: In this paper, we define the action as:

$$A = (SF, TP). \quad (12)$$

In LoRa, the SF's value range is $\{7, 8, 9, 10, 11, 12\}$, and TP's value range is $\{2dBm, 5dBm, 8dBm, 11dBm, 14dBm\}$. Therefore, there exist 30 actions in the action space.

3) *Reward Function*: In this paper, the optimization goal needs to consider throughput, energy efficiency, and reliability simultaneously. Thus, we define the reward function as:

$$R(a, s) = u^{\mathcal{D}_{j,g}} = \alpha u_{QoS}^{\mathcal{D}_{j,g}} + \beta u_{EE}^{\mathcal{D}_{j,g}} + \gamma u_{REL}^{\mathcal{D}_{j,g}}, \quad (13)$$

where α , β , and γ are system weight parameters. Different values indicate different preferences of the slice QoS requirement. For example, a larger γ should be adopted in the URA slice for the sake of better transmission reliability.

B. Deep Deterministic Policy Gradient

In this paper, we introduce the DDPG to learn the optimal slice configuration policy. DDPG is a model-free off-policy RL algorithm that combines the deep Q-Network (DQN) algorithm and deterministic policy gradient (DPG) algorithm [22]. As shown in Fig. 2, it is composed of two neural network components, termed as critic and actor. The actor function $\mu(s|\theta^\mu)$ specifies action a given the current state s of the environments. Critic value function $Q(s, a|\theta^Q)$ specifies a signal (TD Error) to criticize the action made by the actor.

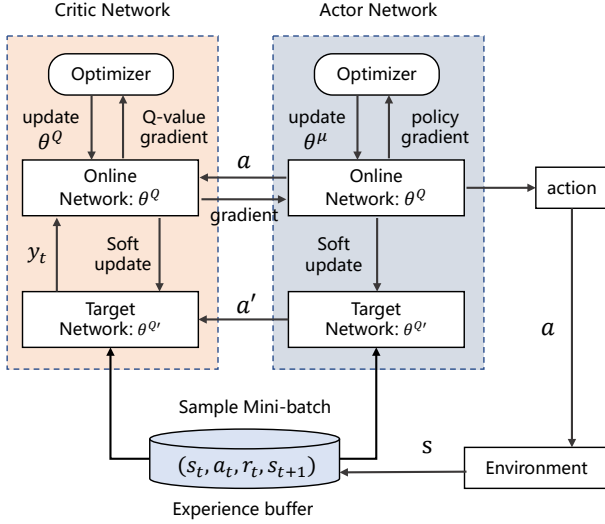


Fig. 2. The DDPG Algorithm.

For the actor-network, the objective function can be described as:

$$J(\theta^\mu) = E_{\theta^\mu}[r_1 + \gamma r_2 + \gamma^2 r_3 + \dots]. \quad (14)$$

The actor-network will update the parameters θ^μ toward the direction of increasing $J(\theta)$. The gradient of the objective $J(\theta^\mu)$ can be expressed as:

$$\frac{\partial J(\theta^\mu)}{\partial \theta^\mu} = E_s \left[\frac{\partial Q(s, a | \theta^Q)}{\partial a} \frac{\partial \mu(s | \theta^\mu)}{\partial \theta^\mu} \right]. \quad (15)$$

For the critic network, it calculates the Q-value of the observation-action pair (s, a) to measure the profit of action a under a specific state s . The update of the critic network θ^Q can be described as:

$$\frac{\partial L(\theta^Q)}{\partial \theta^Q} = E_{s,a,r,s'} [(TargetQ - Q(s, a | \theta^Q)) \frac{\partial Q(s, a | \theta^Q)}{\partial \theta^Q}], \quad (16)$$

where

$$TargetQ = r + \gamma Q'(s', \mu(s' | \theta^{\mu'}) | \theta^{Q'}). \quad (17)$$

Besides, to stabilize the training process, DDPG adopts the Experience Replay and Target Networks scheme.

Experience Reply: Experience Replay is a replay memory technique, where agent's experiences are stored as a tuple of $[s_t, a_t, r_t, s_{t+1}]$ in a replay buffer D . During the training, the RL agent randomly drew a mini-batch of experience from D to train the network. Such storage-sampling act effectively addresses the unstable training problem caused by the auto-correlation among training data [23].

Target Network: During the training, since the learning object constantly changes, the value TD estimations can easily spiral out of control. To mitigate that risk, the target network is introduced. The target network's weights are fixed during the learning process, and periodically reset to the original network's values.

In DDPG, we define the target critic network and target actor-network to calculate the Q-value for the next state in TD-error computations:

$$policy\ network \begin{cases} online : \mu(s | \theta^\mu) : gradient\ update\ \theta^\mu \\ target : \mu(s | \theta^{\mu'}) : soft\ update\ \theta^{\mu'} \end{cases} \quad (18)$$

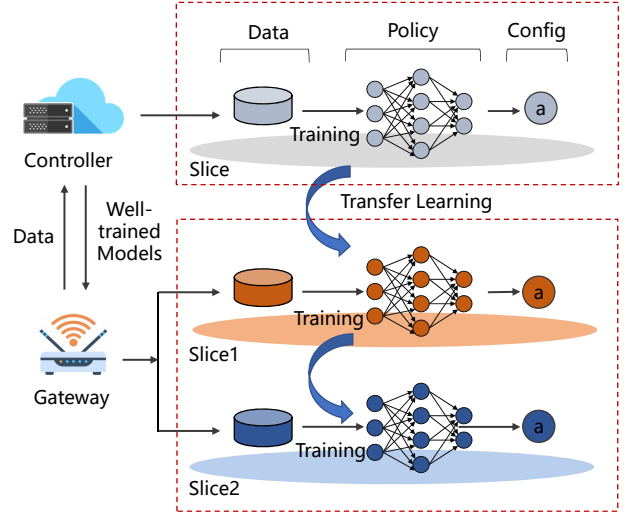


Fig. 3. Transfer Reinforcement Learning

$$Q\ network \begin{cases} online : Q(s, a | \theta^Q) : gradient\ update\ \theta^Q \\ target : Q(s, a | \theta^{Q'}) : soft\ update\ \theta^{Q'} \end{cases} \quad (19)$$

In DDPG, the weights of targets are updated based on the main networks periodically, which can be formulated as:

$$soft\ update : \begin{cases} \theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \\ \theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'} \end{cases} \quad (20)$$

C. Transfer Reinforcement Learning

As discussed above, the DDPG agent can learn from its experience and improve its performance by exploring the environment. However, such learning mechanism has to take a large number of training episodes to converge the optimal value. Especially in our scenario, multiple slice agents are co-existed. Each RL agent has to learn from scratch, and therefore reduce the system utility.

To accelerate the learning process across multiple agents, the transfer learning techniques are integrated into our method to facilitate the learning process. Transfer learning is a machine learning method that the learned models' trained from a task are reused as the starting point for new models on another task [24]. Different from the isolated learning paradigm, transfer learning exploits the knowledge acquired from previous tasks to improve generalization and learning rate about related ones.

Combining with the transfer learning methods, as shown in Fig. 3, we propose a transfer multi-agent Deep Deterministic Policy Gradient (TMDDPG) scheme. At the first stage, the centralized controller gathers the data experience $[s_t, a_t, r_t, s_{t+1}]$ from the distributed gateways to construct a replay buffer and train the model. After convergence, the well-trained networks models will be issued to each gateway and used as the starting point on local slice optimization tasks.



Fig. 4. The Convergence Analysis

Algorithm 1 The TMDDPG based slicing optimization algorithm

```

Import pre-trained model and replay buffer data from SDN
controller
Initialize online network weights  $\theta^Q, \theta^\mu$ 
Initialize target network weights  $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$ 
for step = 1 to maximum episode length do
  Observe initial state  $s_0$ 
  for step = 1 to maximum episode duration do
    Takes an action according to the current policy:
       $a_t = \mu(s_t | \theta^\mu) + \mathcal{N}_t$ 
    Execute action  $a_t$  and receive a new state  $s_{t+1}$ 
    Storage the transition in  $R$ 
    Sample a batch of experiences
       $R \times (s, a, r, s')$  from replay buffer
    Calculate TD target of Q-network:
       $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} | \theta^{\mu'})) | \theta^{Q'}$ 
    Update online Q-network by minimizing the loss:
       $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2$ 
    Update online policy network with:
       $\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a | \theta^Q) \nabla_{\theta^\mu} |_{s=s_i, a=\mu(s_i | \theta^\mu)}$ 
    Update the target networks by:
       $\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$ 
       $\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$ 
  end for
end for

```

Moreover, the experience memory pool of each slice agent is imported by well-trained model experience in the SDN controller.

The slice optimization policy of the TMDDPG algorithm is shown in detail in Algorithm 1.

V. SIMULATION RESULTS

In this section, we present the simulation results to evaluate the validity of the proposed algorithm. Our experiments simulate Ubuntu 16.04 with 32g RAM, Nvidia RTX 2060, and intel i7-10875H. We use OMNET++ for building the LoRaWAN

simulation environment, and PyTorch 1.4.0 for implementing neural networks.

A. Simulation Settings

In our experiment, we simulate a LoRa-based network with 4 gateways (i.e., $G = 4$). The number of slices on each gateway is 3, and the number of industrial IoT device assigned to each slice range from 100 to 1000. We assume that gateways and devices are uniformly distributed in a cell of a 10KM radius. In addition, the number of channels per slice varies from 1 – 3 and each channel Bandwidth is 125kHz. The SF's value range is $\{7, 8, 9, 10, 11, 12\}$, and TP's value range is $\{2dBm, 5dBm, 8dBm, 11dBm, 14dBm\}$. The detailed parameters setting can be found in Table II.

TABLE II
SIMULATION PARAMETERS

Parameter	Value
Number of gateways	4
Number of slice	3
Number of devices per slice	100 to 1000
Number of Channels per slice	1 to 3
Channel Bandwidth	125 kHz
Gateways and devices distribution	Uniform distribution
Training batch size	25000 steps
Spreading Factor	7 to 12
Transmit power	2 to 14 dBm

B. Convergence Analysis

First, we evaluate the convergence of our proposed algorithm. We adopt two other reinforcement learning algorithms, DDPG and DQN, as the baseline algorithms. The configuration of TMDDPG, DDPG, and DQN are shown in Table. III.

As shown in Fig. 4, the learning process of three algorithms is demonstrated. We notice that three algorithms can all converge to a near reward. This demonstrates that reinforcement learning is capable of improving policy performance through interacting with the environment. Also, we notice that TMDDPG exhibits a better convergence compared to the DDPG and the DQN. The TMDDPG can obtain stable reward around 500 episodes, while DQN around 1300 episodes and DDPG around 1700 episodes. This is because TMDDPG acquires knowledge from previous tasks, without having to restart training from the scratch.

TABLE III
TMDDPG & DDPG & DQN CONFIGURATION

Parameter	Value
Episode maximum duration	100s (100 steps)
Maximum episode	20000 episodes
Discount factor (γ)	0.95
Learning rate (α)	0.01
Training batch size	25000 steps
Replay buffer size	100
Minibatch size	10
Hidden layer size	64
Step duration (τ)	1s

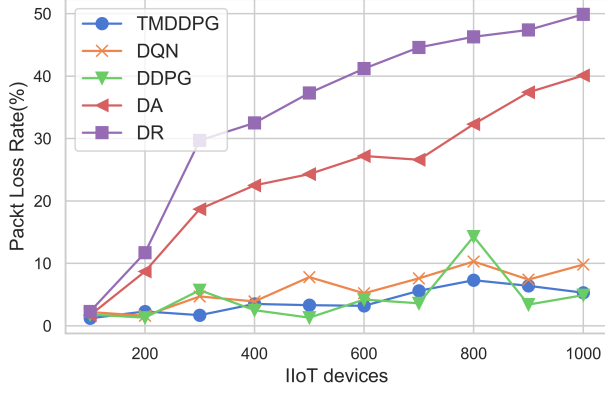


Fig. 5. The Performance Evaluation.

C. Performance Analysis

In this part, we evaluate the algorithm performance in terms of network reliability. Two classical SF-TP adjust algorithm, termed dynamic random (DR) algorithm and dynamic adaptive (DA), are set as the baseline, where the DR configure the SF and TP values randomly, and DA configure the SF and TP values based on link quality with specific SF-TP pair, including (7,2dbm), (8,5dbm), (9,8dbm), (10,11dbm), (11,14dbm) and (12,14dbm).

As shown in Fig. 5, we present the packet loss rate (PLR) of the different algorithms. With the number of devices increasing, the PLR increase. This is mainly caused by the increase of inter-and intra- collision probability with the number of devices increasing. Besides, we notice that reinforcement learning algorithms exhibit better performance than DA and DR algorithms. This demonstrates the validity and effectiveness of our proposed algorithm.

D. Slice Performance Analysis

In this part, we will evaluate the different kinds of slice's performance in terms of delay, energy efficiency, and packet loss rate. As discussed above, we define three types of slices, i.e., UR, RA, and BE. Each type agent adopts different weight parameters in the reward function for meeting the QoS preference, where α indicates the QoS weight, β indicates the energy efficiency weight, and γ indicates the reliability weight. We detail the parameter's value in Table. IV.

TABLE IV
SLICE PARAMETERS CONFIGURATION

Slice	α value	β value	γ value
UR	0.4	0.1	0.5
RA	0.4	0.2	0.4
BE	0.3	0.4	0.3

1) *Delay Evaluation*: Firstly, we evaluate the delay performance of different slices. As shown in Fig. 6, with the number of devices increasing, the delay also increases. This is caused by inter-and intra- interference. With the collision probability increasing, the gateway has to set higher SF and TP values to

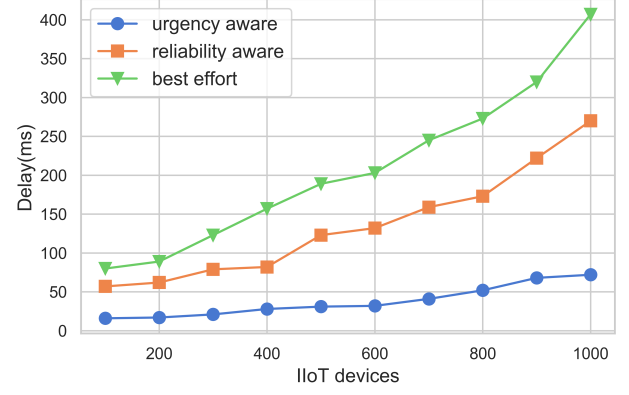


Fig. 6. The Delay in Different Slices.

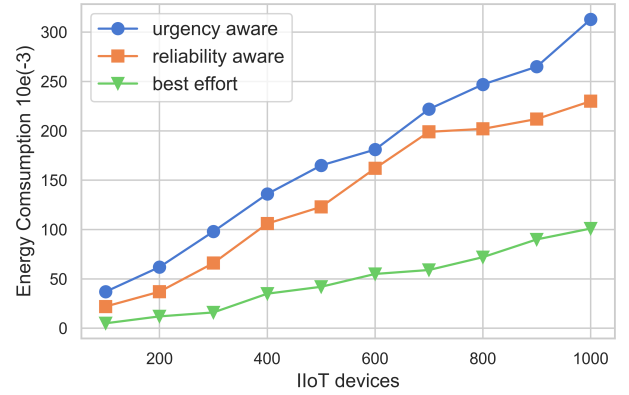


Fig. 7. The Energy Consumption in Different Slices.

enhance system reliability. Hence, the data rate will decrease and the transmission delay will increase.

Also, we notice that the UR and RA's delay is always lower than the BE slice. The reason is that UR and RA slices' reward functions are configured with the higher α and γ . The agent will gain more profit with the higher throughput and lower delay.

2) *Energy Evaluation*: Then, we present the energy consumption of different slices. As shown in Fig. 7, with the number of devices increasing, the total energy consumption also increases. That's because that more participants will bring more energy consumption to the total consumption. Besides, we notice that the energy consumption of UR is always lower than the RA and BE. This is because that UR's reward function adopts a higher value of γ , which forces the agent to take a higher TP value compared to the RA and BE for higher reliability. And increasing SF and TP values will increase energy consumption. In addition, the BE's reward adopts the highest β , which drives the agent to adjust its policy to the lower consumptions direction.

3) *Packet Loss Rate Evaluation*: We plot the PLR performance in Fig. 8. With the devices increasing, the PLR will also increase subsequently. This is because that more packets will be transferred at the same time, and therefore aggravated the inter-and intra- problem. Besides, the PRL of UR is lower

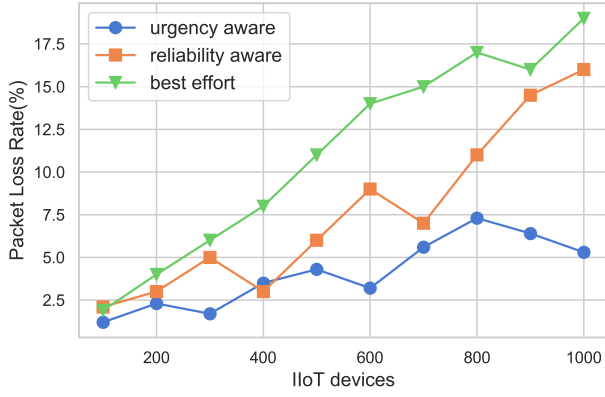


Fig. 8. Packet Loss Rate in Different Slices.

than the RA and BE, and there are obvious reasons for that.

These results demonstrate that the TMDDPG can dynamically optimize slice performance by adjusting the SF and TP parameters configuration. Besides, the reward function design can directly affect the performance in terms of delay, energy efficiency, and reliability. The slice agent can adjust the reward's weights (i.e., α , β , γ) for acquiring different performance preferences.

VI. CONCLUSION

In this paper, we discuss a network slicing architecture over SDN-based LoRaWAN, where an SDN controller divides LoRa gateways' physical resources into multiple virtual networks to provide different QoS guarantees. In addition, considering the limited number of available channels on each LoRa gateway, slices may suffer from performance degradation and resource starvation. To tackle this problem, we propose a DDPG based slice optimization algorithm to search the optimal SF and TP parameters configurations. In addition, to accelerate the learning process across multiple LoRa gateways, a multi-regional intelligent slicing optimization algorithm that utilizes transferred knowledge from network service is proposed. The experimental results show that the TMDDPG can dynamically optimize slice performance by adjusting the SF and TP parameters configurations.

REFERENCES

- [1] G. Han, J. Tu, L. Liu, M. Martínez-García, and C. Choi, "An intelligent signal processing data denoising method for control systems protection in the industrial internet of things," *IEEE Transactions on Industrial Informatics*, 2021.
- [2] A. Lavric and V. Popa, "Internet of things and lora low-power wide-area networks: a survey," in *2017 International Symposium on Signals, Circuits and Systems (ISSCS)*, pp. 1–5. IEEE, 2017.
- [3] G. Han, J. Tu, L. Liu, M. Martínez-García, and Y. Peng, "Anomaly detection based on multidimensional data processing for protecting vital devices in 6g-enabled massive iiot," *IEEE Internet of Things Journal*, vol. 8, no. 7, pp. 5219–5229, 2021.
- [4] S. Wijethilaka and M. Liyanage, "Survey on network slicing for internet of things realization in 5g networks," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 957–994, 2021.
- [5] M. Bor, J. E. Vidler, and U. Roedig, "Lora for the internet of things," 2016.
- [6] J. Wang, C. Jiang, H. Zhang, Y. Ren, K.-C. Chen, and L. Hanzo, "Thirty years of machine learning: The road to pareto-optimal wireless networks," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 1472–1514, 2020.
- [7] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on knowledge and data engineering*, vol. 22, no. 10, pp. 1345–1359, 2009.
- [8] I. Afolabi, T. Taleb, K. Samdanis, A. Ksentini, and H. Flinck, "Network slicing and softwarization: A survey on principles, enabling technologies, and solutions," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 3, pp. 2429–2453, 2018.
- [9] S. Dawaliby, A. Bradai, and Y. Pousset, "Adaptive dynamic network slicing in lora networks," *Future generation computer systems*, vol. 98, pp. 697–707, 2019.
- [10] S. Dawaliby, A. Bradai, and Y. Pousset, "Distributed network slicing in large scale iot based on coalitional multi-game theory," *IEEE Transactions on Network and Service Management*, vol. 16, no. 4, pp. 1567–1580, 2019.
- [11] S. Dawaliby, A. Bradai, and Y. Pousset, "Joint slice-based spreading factor and transmission power optimization in lora smart city networks," *Internet of Things*, p. 100121, 2019.
- [12] R. Kufakunesu, G. P. Hancke, and A. M. Abu-Mahfouz, "A survey on adaptive data rate optimization in lorawan: Recent solutions and major challenges," *Sensors*, vol. 20, no. 18, p. 5044, 2020.
- [13] I. Ilahi, M. Usama, M. O. Farooq, M. U. Janjua, and J. Qadir, "Intelligent resource allocation in dense lora networks using deep reinforcement learning," *arXiv preprint arXiv:2012.11867*, 2020.
- [14] G. Park, W. Lee, and I. Joe, "Network resource optimization with reinforcement learning for low power wide area networks," *EURASIP Journal on Wireless Communications and Networking*, vol. 2020, no. 1, pp. 1–20, 2020.
- [15] B. Reynders, Q. Wang, P. Tuset-Peiro, X. Vilajosana, and S. Pollin, "Improving reliability and scalability of lorawans through lightweight scheduling," *IEEE Internet of Things Journal*, vol. 5, no. 3, pp. 1830–1842, 2018.
- [16] B. K. Al-Shammari, N. Al-Aboudy, and H. S. Al-Rawashidy, "Iot traffic management and integration in the qos supported network," *IEEE Internet of Things Journal*, vol. 5, no. 1, pp. 352–370, 2017.
- [17] K. Benzekki, A. El Fergougui, and A. Elbelrhiti Elalaoui, "Software-defined networking (sdn): a survey," *Security and communication networks*, vol. 9, no. 18, pp. 5803–5833, 2016.
- [18] C. Qiu, H. Yao, C. Jiang, S. Guo, and F. Xu, "Cloud computing assisted blockchain-enabled internet of things," *IEEE Transactions on Cloud Computing*, 2019.
- [19] O. Georgiou and U. Raza, "Low power wide area network analysis: Can lora scale?" *IEEE Wireless Communications Letters*, vol. 6, no. 2, pp. 162–165, 2017.
- [20] K. Xue, B. Zhu, Q. Yang, N. Gai, D. S. Wei, and N. Yu, "Inpptd: A lightweight incentive-based privacy-preserving truth discovery for crowdsensing systems," *IEEE Internet of Things Journal*, vol. 8, no. 6, pp. 4305–4316, 2020.
- [21] T. Mai, H. Yao, N. Zhang, L. Xu, M. Guizani, and S. Guo, "Cloud mining pool aided blockchain-enabled internet of things: An evolutionary game approach," *IEEE Transactions on Cloud Computing*, pp. 1–1, 2021.
- [22] Y. He, G. Han, J. Jiang, H. Wang, and M. Martínez-García, "A trust update mechanism based on reinforcement learning in underwater acoustic sensor networks," *IEEE Transactions on Mobile Computing*, 2020.
- [23] B. Eysenbach, R. Salakhutdinov, and S. Levine, "Search on the replay buffer: Bridging planning and reinforcement learning," *arXiv preprint arXiv:1906.05253*, 2019.
- [24] M. E. Taylor and P. Stone, "Transfer learning for reinforcement learning domains: A survey," *Journal of Machine Learning Research*, vol. 10, no. 7, 2009.



Tianle Mai is pursuing his Ph.D. degree in the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing. His research interests include future network architecture, network artificial intelligence, multi-agent system, space-terrestrial integrated network, network resource allocation and dedicated networks.



Haipeng Yao (M'16, SM'20) is an Associate Professor in Beijing University of Posts and Telecommunications. Haipeng Yao received his Ph.D. in the Department of Telecommunication Engineering at University of Beijing University of Posts and Telecommunications in 2011. His research interests include future network architecture, network artificial intelligence, networking, space-terrestrial integrated network, network resource allocation and dedicated networks. He has published more than 100 papers in prestigious peer-reviewed journals and

conferences. Dr. Yao has served as an Editor of IEEE Network, IEEE Access, and a Guest Editor of IEEE Open Journal of the Computer Society and Springer Journal of Network and Systems Management. He has also served as a member of the technical program committee as well as the Symposium Chair for a number of international conferences, including IWCMC 2019 Symposium Chair, ACM TUR-C SIGSAC2020 Publication Chair.



Ni Zhang currently serves at the Sixth Research Institute of China Electronic Corporation, Beijing, China. He received his Ph. D. at the Institute of Computing Technology, Chinese Academy of Sciences in 2007. His research interests are in the area of future Internet architecture, network security and artificial intelligence.



Wenji He is currently an undergraduate student at Beijing University of Posts and Telecommunications. Her research interests are in the areas of computer working and the security of the Internet.



Dong Guo received the B.S. degree in communication engineering with the School of information and Communication Engineering, Beijing University of Posts and Telecommunication, Beijing, China, in 2012, and the Ph.D. degree in optical engineering from the Beijing University of Posts and Telecommunication, in 2019. Since 2019, he has been a post-doctoral researcher with the School of Information Electronics, Beijing Institute of Technology, Beijing. His research interests include optical fiber communication, forward error correction, digital signal processing and high-order modulation.



Mohsen Guizani (S'85-M'89-SM'99-F'09) received the BS (with distinction), MS and PhD degrees in Electrical and Computer engineering from Syracuse University, Syracuse, NY, USA, in 1984, 1986, and 1990, respectively. He is currently a Professor and Associate Provost at MBZUAI, Abu Dhabi, UAE. Previously, he worked in different institutions in the USA: University of Idaho, Western Michigan University, University of West Florida, University of Missouri, University of Colorado-Boulder, and Syracuse University. His research inter-

ests include wireless communications and mobile computing, applied machine learning, cloud computing, security and its application to healthcare systems. He was elevated to the IEEE Fellow in 2009. He was listed as a Clarivate Analytics Highly Cited Researcher in Computer Science in 2019, 2020 and 2021. Dr. Guizani has won several research awards including the 2015 IEEE Communications Society Best Survey Paper Award as well 4 Best Paper Awards from ICC and Globecom Conferences. He is the author of nine books and more than 800 publications. He is also the recipient of the 2017 IEEE Communications Society Wireless Technical Committee (WTC) Recognition Award, the 2018 AdHoc Technical Committee Recognition Award, and the 2019 IEEE Communications and Information Security Technical Recognition (CISTC) Award. He served as the Editor-in-Chief of IEEE Network and is currently serving on the Editorial Boards of many IEEE Transactions and Magazines. He was the Chair of the IEEE Communications Society Wireless Technical Committee and the Chair of the TAOS Technical Committee. He served as the IEEE Computer Society Distinguished Speaker and is currently the IEEE ComSoc Distinguished Lecturer.